

Reviewer Pack — Ehrhart Coefficient Growth Bounds Tseitin Resolution Length

Entry 05f7ca79b6ba · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 12:25:07 UTC

Ehrhart Coefficient Growth Bounds Tseitin Resolution Length

Entry ID: 05f7ca79b6ba **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 12:25:07 UTC

1. Conjecture

Field A (mathematical branch): Ehrhart Theory **Field B** (complexity object): Resolution Length of Tseitin Formulas

Statement:

For a Tseitin formula on a connected graph G with n vertices, the resolution length is $\geq 2^{\{\Omega(v(G))\}}$, where $v(G)$ is the coefficient of x^{n-2} in the Ehrhart polynomial of the cycle polytope of G . For graphs with $v(G) = O(1)$, the resolution length is polynomial.

Rationale (proposer’s reasoning):

Ehrhart coefficients encode the combinatorial complexity of integer-point counts in polytopes. By linking the asymptotic growth of these coefficients to resolution steps, we may uncover geometric obstructions to efficient refutations in Tseitin formulas, leveraging the interplay between algebraic combinatorics and proof complexity.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f2375117c510fc54

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A, b):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
            for j in range(i+1, m):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
                b[j] -= factor * b[i]
        x = [0] * n
        for i in range(m-1, -1, -1):
            x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
        return x

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def determinant(A):
        if len(A) == 1:
            return A[0][0]
        det = 0
        for i in range(len(A)):
            submatrix = [row[:i] + row[i+1:] for row in A[1:]]
```

```

        det += ((-1) ** i) * A[0][i] * determinant(submatrix)
    return det

def inverse(A):
    n = len(A)
    I = [[Fraction(1, 1) if i == j else Fraction(0, 1) for j in range(n)] for i in range(n)]
    Augmented = [A[i] + I[i] for i in range(n)]
    for i in range(n):
        factor = Augmented[i][i]
        for j in range(n * 2):
            Augmented[i][j] /= factor
        for j in range(n):
            if i != j:
                factor = Augmented[j][i]
                for k in range(n * 2):
                    Augmented[j][k] -= factor * Augmented[i][k]
    return [row[n:] for row in Augmented]

def solve_linear_system(A, b):
    A_inv = inverse(A)
    x = matrix_multiply(A_inv, b)
    return x

def cycle_polytope_vertex_enumeration(n):
    if n == 3:
        return [[0, 0], [1, 0], [0, 1]]
    elif n == 4:
        return [[0, 0], [1, 0], [1, 1], [0, 1]]
    else:
        return []

def ehrhart_polynomial_coefficient(polytope_vertices):
    n = len(polytope_vertices[0])
    A = [[0] * (n + 1) for _ in range(n + 1)]
    b = [0] * (n + 1)
    for vertex in polytope_vertices:
        A[0][n] += 1
        for i in range(n):
            A[i+1][i] = 1
            A[i+1][n] -= vertex[i]
            b[i+1] += vertex[i]
    x = solve_linear_system(A, b)
    return x[n]

def resolution_length(graph):
    # Placeholder for actual DPLL-based solver implementation
    return random.randint(100, 1000)

def tseitin_formula_resolution_length(n):
    graph = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    polytope_vertices = cycle_polytope_vertex_enumeration(n)
    if not polytope_vertices:
        return None
    v_G = ehrhart_polynomial_coefficient(polytope_vertices)
    if v_G == 0:
        return resolution_length(graph)

```

```

        else:
            return None

n = random.choice([5, 10, 15, 20, 30, 40])
result = tseitin_formula_resolution_length(n)
if result is None:
    return {
        "metric_name": "resolution_length",
        "metric_value": -1,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

v_G = ehrhart_polynomial_coefficient(cycle_polytope_vertex_enumeration(n))
if v_G == 0:
    return {
        "metric_name": "resolution_length",
        "metric_value": result,
        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    return {
        "metric_name": "resolution_length",
        "metric_value": result,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Graph with n={n}, v(G)={v_G}"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["instances_tested"] > 0]
    conjecture_holds_count = sum(1 for r in results if r["conjecture_holds"])

    mean = sum(metric_values) / len(metric_values)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = conjecture_holds_count / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Graph with v(G) > 0\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
njecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'resolution_length', 'metric_value': -1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'resolution_length', 'metric_value': -1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'resolution_length', 'metric_value': -1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'resolution_length', 'metric_value': -1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'resolution_length', 'metric_value': -1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'resolution_length', 'metric_value': -1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'resolution_length', 'metric_value': -1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'resolution_length', 'metric_value': -1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'resolution_length', 'metric_value': -1, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5478ca5d.py", line 162, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5478ca5d.py", line 162, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
KeyError: 'seed'
```

8. Critic adversarial review

Critic verdict: CHALLENGE
Critic reasoning:
skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE
Reasoning:
Test crashed with KeyError ‘seed’ before producing valid data; conjecture_holds is unreliable | next: Implement error handling for seed tracking and re-run tests with concrete graph instances

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	45230
2	propose	ollama_remote	qwen3:8b	0	0	55113
3	preregistration	ollama_remote	qwen3:8b	0	0	33903
4	novelty	ollama_remote	qwen3:8b	0	0	27949
5	novelty	ollama_remote	qwen3:8b	0	0	20469
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15775
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9277

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16121
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17867
10	judge	ollama_remote	qwen3:8b	0	0	22071

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 263775 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/05f7ca79b6ba.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/05f7ca79b6ba.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/05f7ca79b6ba.tar.gz (if generated)